

Assignment 2 b**Create Docker Container Environment (NVIDIA Docker or any other).****DOCKER:**

Docker is a popular containerization platform that enables developers to package their applications and their dependencies into a container. These containers can then be deployed on any system with Docker installed, making it a highly portable and flexible solution for deploying applications.

NVIDIA Docker is an extension of the Docker platform that allows developers to run GPU accelerated applications inside Docker containers. This is particularly useful for machine learning and data science applications, as these often require access to specialized hardware like GPUs.

FEATURES OF DOCKER

- Docker has the ability to reduce the size of development by providing a smaller footprint of the operating system via containers.
- With containers, it becomes easier for teams across different units, such as development, QA and Operations to work seamlessly across applications.
- You can deploy Docker containers anywhere, on any physical and virtual machines and even on the cloud.
- Since Docker containers are pretty lightweight, they are very easily scalable.

COMPONENTS:

Docker has the following components

- Docker for Mac – It allows one to run Docker containers on the Mac OS.
- Docker for Linux - It allows one to run Docker containers on the Linux OS.
- Docker for Windows - It allows one to run Docker containers on the Windows OS.
- Docker Engine – It is used for building Docker images and creating Docker containers.
- Docker Hub – This is the registry which is used to host various Docker images.
- Docker Compose – This is used to define applications using multiple Docker containers.

DOCKER – INSTALLING DOCKER ON LINUX

To install Docker, we need to follow the steps given below.

Step 1:

Before installing Docker, you first have to ensure that you have the right Linux kernel version running. Docker is only designed to run on Linux kernel version 3.8 and higher.

We can do this by running the following command:

uname This method returns the system information about the Linux system

Example

```
uname -a
```

Output

When we run above command, we will get the following result:

```
demo@ubuntu:~$ uname -a
Linux ubuntu 4.2.0-27-generic #32~14.04.1-Ubuntu SMP Fri Jan 22 15:32:27 UTC 201
6 i686 i686 i686 GNU/Linux
demo@ubuntu:~$ _
```

Step 2: You need to update the OS with the latest packages, which can be done via the following command:

```
apt-get
```

This method installs packages from the Internet on to the Linux system.

Syntax

```
sudo apt-get update
```

Output

When we run the above command, we will get the following result:

```
Hit http://us.archive.ubuntu.com trusty-backports/universe Sources
Hit http://us.archive.ubuntu.com trusty-backports/multiverse Sources
Hit http://us.archive.ubuntu.com trusty-backports/main i386 Packages
Hit http://us.archive.ubuntu.com trusty-backports/restricted i386 Packages
Hit http://us.archive.ubuntu.com trusty-backports/universe i386 Packages
Hit http://us.archive.ubuntu.com trusty-backports/multiverse i386 Packages
Hit http://us.archive.ubuntu.com trusty-backports/main Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/multiverse Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/restricted Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/universe Translation-en
Hit http://us.archive.ubuntu.com trusty Release
Hit http://us.archive.ubuntu.com trusty/main Sources
Hit http://us.archive.ubuntu.com trusty/restricted Sources
Hit http://us.archive.ubuntu.com trusty/universe Sources
Hit http://us.archive.ubuntu.com trusty/multiverse Sources
Hit http://us.archive.ubuntu.com trusty/main i386 Packages
Hit http://us.archive.ubuntu.com trusty/restricted i386 Packages
Hit http://us.archive.ubuntu.com trusty/universe i386 Packages
Hit http://us.archive.ubuntu.com trusty/multiverse i386 Packages
Hit http://us.archive.ubuntu.com trusty/main Translation-en
Hit http://us.archive.ubuntu.com trusty/multiverse Translation-en
Hit http://us.archive.ubuntu.com trusty/restricted Translation-en
Hit http://us.archive.ubuntu.com trusty/universe Translation-en
Ign http://us.archive.ubuntu.com trusty/main Translation-en_US
Ign http://us.archive.ubuntu.com trusty/multiverse Translation-en_US
Ign http://us.archive.ubuntu.com trusty/restricted Translation-en_US
Ign http://us.archive.ubuntu.com trusty/universe Translation-en_US
Fetched 3,906 kB in 21s (184 kB/s)
Reading package lists... Done
demo@ubuntu:~$
```

This command will connect to the internet and download the latest system packages for Ubuntu.

Step 3:

The next step is to install the necessary certificates that will be required to work with the Docker site later on to download the necessary Docker packages. It can be done with the following command:

```
sudo apt-get install apt-transport-https ca-certificates
```

```
demo@ubuntudemo:~$ sudo apt-get install apt-transport-https ca-certificates
Reading package lists... Done
Building dependency tree
Reading state information... Done
The following packages will be upgraded:
  apt-transport-https ca-certificates
2 upgraded, 0 newly installed, 0 to remove and 105 not upgraded.
Need to get 215 kB of archives.
After this operation, 8,192 B disk space will be freed.
Get:1 http://us.archive.ubuntu.com/ubuntu/ trusty-updates/main apt-transport-
ps amd64 1.0.1ubuntu2.15 [25.0 kB]
Get:2 http://us.archive.ubuntu.com/ubuntu/ trusty-updates/main ca-certificat
11 20160104ubuntu0.14.04.1 [190 kB]
Fetched 215 kB in 1s (152 kB/s)
Preconfiguring packages ...
(Reading database ... 57694 files and directories currently installed.)
Preparing to unpack .../apt-transport-https_1.0.1ubuntu2.15_amd64.deb ...
Unpacking apt-transport-https (1.0.1ubuntu2.15) over (1.0.1ubuntu2.11) ...
Preparing to unpack .../ca-certificates_20160104ubuntu0.14.04.1_all.deb ...
Unpacking ca-certificates (20160104ubuntu0.14.04.1) over (20141019ubuntu0.14.
1) ...
Processing triggers for man-db (2.6.7.1-1ubuntu1) ...
Setting up apt-transport-https (1.0.1ubuntu2.15) ...
Setting up ca-certificates (20160104ubuntu0.14.04.1) ...
Processing triggers for ca-certificates (20160104ubuntu0.14.04.1) ...
Updating certificates in /etc/ssl/certs... 19 added, 19 removed; done.
Running hooks in /etc/ca-certificates/update.d....done.
demo@ubuntudemo:~$
```


Step 4: The next step is to add the new GPG key. This key is required to ensure that a is encrypted when downloading the necessary packages for Docker.

The following command will download the key with the 58118E89F3A912897C070ADBF76221572C52609D from the **keyserver** hkp://ha.pool.sks-keyservers.net:80 and adds it to the **adv** keychain. Please note that this particular required to download the necessary Docker packages.

```
sudo apt-key adv \
    --keyserver hkp://ha.pool.sks-keyservers.net:80 \
    --recv-keys 58118E89F3A912897C070ADBF76221572C52609D
```

```
demo@ubuntu:~$ sudo apt-key adv --keyserver hkp://ha.pool.sks-keyservers.net:80 \
--recv-keys 58118E89F3A912897C070ADBF76221572C52609D
Executing: gpg --ignore-time-conflict --no-options --no-default-keyring --homedir /tmp/tmp.KcaZ3WlmGt --no-auto-check-trustdb --trust-model always --keyring /etc/apt/trusted.gpg --primary-keyring /etc/apt/trusted.gpg --keyserver hkp://ha.pool.sks-keyservers.net:80 --recv-keys 58118E89F3A912897C070ADBF76221572C52609D
gpg: requesting key 2C52609D from hkp server ha.pool.sks-keyservers.net
gpg: key 2C52609D: public key "Docker Release Tool (releasedocker) <docker@docker.com>" imported
gpg: Total number processed: 1
gpg: imported: 1 (RSA: 1)
demo@ubuntu:~$
```

Step 5: Next, depending on the version of Ubuntu you have, you will need to add the relevant site to the **docker.list** for the **apt package manager**, so that it will be able to detect the Docker packages from the Docker site and download them accordingly.

- Precise 12.04 (LTS) – deb [https://apt.dockerproject.org/repo ubuntu-precise](https://apt.dockerproject.org/repo/ubuntu-precise) main
- Trusty 14.04 (LTS) – deb [https://apt.dockerproject.org/repo ubuntu-trusty](https://apt.dockerproject.org/repo/ubuntu-trusty) main
- Wily 15.10 – deb [https://apt.dockerproject.org/repo ubuntu-wily](https://apt.dockerproject.org/repo/ubuntu-wily) main
- Xenial 16.04 (LTS) – deb [https://apt.dockerproject.org/repo ubuntu-xenial](https://apt.dockerproject.org/repo/ubuntu-xenial) main

Since our OS is Ubuntu 14.04, we will use the Repository name as “deb [https://apt.dockerproject.org/repo ubuntu-trusty](https://apt.dockerproject.org/repo/ubuntu-trusty) main”

And then, we will need to add this repository to the **docker.list** as mentioned above.

```
echo "deb https://apt.dockerproject.org/repo ubuntu-trusty main" | sudo tee /etc/apt/sources.list.d/docker.list
```

```
demo@ubuntu:~$ echo "deb https://apt.dockerproject.org/repo ubuntu-trusty main" | sudo tee /etc/apt/sources.list.d/docker.list
deb https://apt.dockerproject.org/repo ubuntu-trusty main
demo@ubuntu:~$ _
```

Step 6: Next, we issue the apt-get update command to update the packages on the Ubuntu system.

```
Hit http://us.archive.ubuntu.com trusty-backports/multiverse 1500 Packages
Hit http://us.archive.ubuntu.com trusty-backports/main Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/multiverse Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/restricted Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/universe Translation-en
Hit http://us.archive.ubuntu.com trusty Release
Hit http://us.archive.ubuntu.com trusty/main Sources
Hit http://us.archive.ubuntu.com trusty/restricted Sources
Hit http://us.archive.ubuntu.com trusty/universe Sources
Hit http://us.archive.ubuntu.com trusty/multiverse Sources
Hit http://us.archive.ubuntu.com trusty/main amd64 Packages
Hit http://us.archive.ubuntu.com trusty/restricted amd64 Packages
Hit http://us.archive.ubuntu.com trusty/universe amd64 Packages
Hit http://us.archive.ubuntu.com trusty/multiverse amd64 Packages
Hit http://us.archive.ubuntu.com trusty/main i386 Packages
Hit http://us.archive.ubuntu.com trusty/restricted i386 Packages
Hit http://us.archive.ubuntu.com trusty/universe i386 Packages
Hit http://us.archive.ubuntu.com trusty/multiverse i386 Packages
Hit http://us.archive.ubuntu.com trusty/main Translation-en
Hit http://us.archive.ubuntu.com trusty/multiverse Translation-en
Hit http://us.archive.ubuntu.com trusty/restricted Translation-en
Hit http://us.archive.ubuntu.com trusty/universe Translation-en
Ign http://us.archive.ubuntu.com trusty/main Translation-en_US
Ign http://us.archive.ubuntu.com trusty/multiverse Translation-en_US
Ign http://us.archive.ubuntu.com trusty/restricted Translation-en_US
Ign http://us.archive.ubuntu.com trusty/universe Translation-en_US
Fetched 3,333 kB in 36s (90.8 kB/s)
Reading package lists... Done
demo@ubuntudemo:~$
```

Step 7: If you want to verify that the package manager is pointing to the right repository, you can do it by issuing the **apt-cache command**.

```
apt-cache policy docker-engine
```

In the output, you will get the link to <https://apt.dockerproject.org/repo/>

```

1.8.2-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.8.1-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.8.0-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.7.1-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.7.0-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.6.2-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.6.1-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.6.0-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
1.5.0-0~trusty 0
500 https://apt.dockerproject.org/repo/ ubuntu-trusty/main amd64 Packa
s
demo@ubuntudemo:~$

```

Step 8: Issue the **apt-get update** command to ensure all the packages on the local system are up to date.

```

Hit http://us.archive.ubuntu.com trusty-backports/main Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/multiverse Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/restricted Translation-en
Hit http://us.archive.ubuntu.com trusty-backports/universe Translation-en
Hit http://us.archive.ubuntu.com trusty Release
Hit http://us.archive.ubuntu.com trusty/main Sources
Hit http://us.archive.ubuntu.com trusty/restricted Sources
Hit http://us.archive.ubuntu.com trusty/universe Sources
Hit http://us.archive.ubuntu.com trusty/multiverse Sources
Hit http://us.archive.ubuntu.com trusty/main amd64 Packages
Hit http://us.archive.ubuntu.com trusty/restricted amd64 Packages
Hit http://us.archive.ubuntu.com trusty/universe amd64 Packages
Hit http://us.archive.ubuntu.com trusty/multiverse amd64 Packages
Hit http://us.archive.ubuntu.com trusty/main i386 Packages
Hit http://us.archive.ubuntu.com trusty/restricted i386 Packages
Hit http://us.archive.ubuntu.com trusty/universe i386 Packages
Hit http://us.archive.ubuntu.com trusty/multiverse i386 Packages
Hit http://us.archive.ubuntu.com trusty/main Translation-en
Hit http://us.archive.ubuntu.com trusty/multiverse Translation-en
Hit http://us.archive.ubuntu.com trusty/restricted Translation-en
Hit http://us.archive.ubuntu.com trusty/universe Translation-en
Ign http://us.archive.ubuntu.com trusty/main Translation-en_US
Ign http://us.archive.ubuntu.com trusty/multiverse Translation-en_US
Ign http://us.archive.ubuntu.com trusty/restricted Translation-en_US
Ign http://us.archive.ubuntu.com trusty/universe Translation-en_US
Fetched 30.2 kB in 15s (1,980 B/s)
Reading package lists... Done
demo@ubuntudemo:~$

```

Step 9: For Ubuntu Trusty, Wily, and Xenial, we have to install the linux-image-extra-* kernel packages, which allows one to use the **aufs storage driver**. This driver is used by the newer versions of Docker.

It can be done by using the following command:

```
sudo apt-get install linux-image-extra-$(uname -r) linux-image-extra-virtual
```

```
generating grub configuration file ...
Found linux image: /boot/vmlinuz-4.2.0-27-generic
Found initrd image: /boot/initrd.img-4.2.0-27-generic
Found linux image: /boot/vmlinuz-3.13.0-105-generic
Found initrd image: /boot/initrd.img-3.13.0-105-generic
Found memtest86+ image: /memtest86+.elf
Found memtest86+ image: /memtest86+.bin
done
Setting up linux-image-extra-3.13.0-105-generic (3.13.0-105.152) ...
run-parts: executing /etc/kernel/postinst.d/apt-auto-removal 3.13.0-105-generic
/boot/vmlinuz-3.13.0-105-generic
run-parts: executing /etc/kernel/postinst.d/initramfs-tools 3.13.0-105-generic
boot/vmlinuz-3.13.0-105-generic
update-initramfs: Generating /boot/initrd.img-3.13.0-105-generic
run-parts: executing /etc/kernel/postinst.d/update-notifier 3.13.0-105-generic
boot/vmlinuz-3.13.0-105-generic
run-parts: executing /etc/kernel/postinst.d/zz-update-grub 3.13.0-105-generic
boot/vmlinuz-3.13.0-105-generic
Generating grub configuration file ...
Found linux image: /boot/vmlinuz-4.2.0-27-generic
Found initrd image: /boot/initrd.img-4.2.0-27-generic
Found linux image: /boot/vmlinuz-3.13.0-105-generic
Found initrd image: /boot/initrd.img-3.13.0-105-generic
Found memtest86+ image: /memtest86+.elf
Found memtest86+ image: /memtest86+.bin
done
Setting up linux-image-generic (3.13.0.105.113) ...
Setting up linux-image-extra-virtual (3.13.0.105.113) ...
demo@ubuntu:~$
```

Step 10: The final step is to install Docker and we can do this with the following command:

```
sudo apt-get install -y docker-engine
```

Here, **apt-get** uses the install option to download the Docker-engine image from the Docker website and get Docker installed.

The Docker-engine is the official package from the Docker Corporation for Ubuntu-based systems.

```

Selecting previously unselected package liberror-perl.
Preparing to unpack .../liberror-perl_0.17-1.1_all.deb ...
Unpacking liberror-perl (0.17-1.1) ...
Selecting previously unselected package git-man.
Preparing to unpack .../git-man_1:3a1.9.1-1ubuntu0.3_all.deb ...
Unpacking git-man (1:1.9.1-1ubuntu0.3) ...
Selecting previously unselected package git.
Preparing to unpack .../git_1:3a1.9.1-1ubuntu0.3_amd64.deb ...
Unpacking git (1:1.9.1-1ubuntu0.3) ...
Selecting previously unselected package cgroup-lite.
Preparing to unpack .../cgroup-lite_1.9_all.deb ...
Unpacking cgroup-lite (1.9) ...
Processing triggers for man-db (2.6.7.1-1ubuntu1) ...
Processing triggers for ureadahead (0.100.0-16) ...
ureadahead will be reprofiled on next reboot
Setting up libltdl7:amd64 (2.4.2-1.7ubuntu1) ...
Setting up libsystemd-journal0:amd64 (204-5ubuntu20.20) ...
Setting up aufs-tools (1:3.2+20130722-1.1) ...
Setting up docker-engine (1.12.3-0~trusty) ...
docker start/running, process 22612
Setting up liberror-perl (0.17-1.1) ...
Setting up git-man (1:1.9.1-1ubuntu0.3) ...
Setting up git (1:1.9.1-1ubuntu0.3) ...
Setting up cgroup-lite (1.9) ...
cgroup-lite start/running
Processing triggers for libc-bin (2.19-0ubuntu6.7) ...
Processing triggers for ureadahead (0.100.0-16) ...
demo@buntudemo:~$

```

Creating a Docker container environment typically involves several steps :

Installing Docker: To create a Docker container, you must first install Docker on your system. You can follow the installation instructions for your operating system on the Docker website.

Creating a Dockerfile: A Dockerfile is a text file that contains instructions for building a Docker image. This includes installing dependencies, setting environment variables, and copying files into the container. You can create a Dockerfile in any text editor, such as Notepad or Visual Studio Code.

Building the Docker image: Once you have created a Dockerfile, you can use the "docker build" command to build a Docker image from it. This process involves downloading any necessary dependencies and configuring the container.

Running the Docker container: After the Docker image is built, you can use the "docker run" command to start a new container from the image. This will launch your application in a containerized environment.

If you are using NVIDIA Docker, you will need to follow additional steps to configure the container for GPU support. This typically involves installing the NVIDIA drivers and the NVIDIA Docker runtime on your system.

Here is an example Dockerfile for a Python machine learning application using TensorFlow and NVIDIA Docker:

```
FROM nvidia/cuda:11.0-base

RUN apt-get update && apt-get install -y \
python3 \ python3-pip \ python3-dev \
build-essential \ libglib2.0-0 \ libsm6 \
libxext6 \
    libxrender-dev

RUN pip3 install tensorflow-gpu==2.5.0

WORKDIR /app

COPY . /app

CMD ["python3", "myapp.py"]
```

This Dockerfile installs the necessary dependencies for running a Python machine learning application with TensorFlow and NVIDIA GPU support. Once you have created the Dockerfile, you can build and run the Docker container using the following command

```
docker build -t myapp .
docker run --gpus all -it myapp
```

This will build a Docker image named "myapp" and run a new container from it, with GPU support enabled.

Conclusion:

Using Docker containers, including NVIDIA Docker, for machine learning and data science applications can improve portability, reproducibility, scalability, and security.

It can also simplify the deployment process and make it easier to share applications with others.